

COMS W4156 Advanced Software Engineering (ASE)

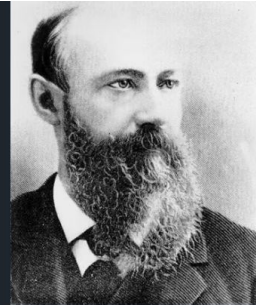
October 10, 2024

On this day ...



Billiard Ball Patented

1865 John Wesley Hyatt patents the billiard ball made from cellulose nitrate (celluloid) in Albany, New York (patent US50359A)



Inventor

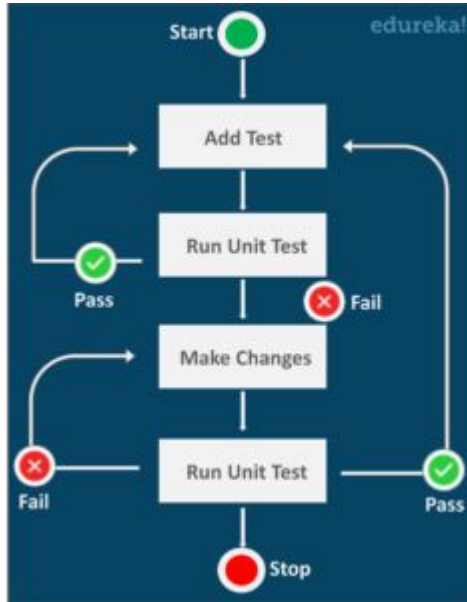
John Wesley Hyatt

Agenda

1. Unit Testing
2. Test Doubles
3. Unit Testing/Mocking demo: Madeleine Lerner
4. Deadlines



Unit Testing



What is a “unit” in unit testing? The smallest piece of code that can reasonably be *isolated* for execution: method, function, procedure, subroutine, etc. but normally not a statement or expression (why not?)

Testing modules (classes) and other closely related assemblies of such units is typically considered part of unit testing, but will be discussed next week with integration testing

“Integration testing” involves testing interfaces between modules and subsystems as well as interactions with external resources such as files, databases and devices

Example Unit Tests

```
public class TestBadCode {  
    public int calculateFoo(int x, int y, boolean increment){  
        if(increment)  
            x++;  
            x *=2;  
        x += y;  
        return x;  
    }  
  
    @Test  
    public void test() {  
        assertEquals(calculateFoo(3, 5, true), 13);  
        assertEquals(calculateFoo(3, 5, false), 8);  
    }  
}
```

calculateFoo is the “unit under test”

This is one test group consisting of two tests, with inputs { 3, 5, true } and { 3, 5, false }

Note this is Java code, not Python.
Which of these tests pass?

Choosing Inputs for Unit Tests

Baseline - one typical valid input (or set of inputs), one atypical valid input, one invalid input value (and one invalid input format if applicable) for every non-trivial unit

Better - Baseline plus one input somewhere in middle of every equivalence partition plus 2-3 inputs at every boundary between equivalence partitions (exactly at boundary, one just above, one just below)

Best? - Better plus “cover” every branch both true and false conditions, plus every loop 0, 1, 2, and many/max (if known) times around loop



Running the Tests

Now that we have chosen some inputs for one or more units under test, how do we run the tests?



Let's say we have a function that takes all its inputs as parameters and provides one output as its return value: write code that calls the function with the input parameters, receives the output, and then checks the output

- The “manual” version is to create a tiny `main()` program just to call the function under test. Don't do it! 🤡 Instead use a unit testing framework

If the function reads/writes local shared state, such as class variables, an object and its instance variables, or an in-memory data structure like a stack or a tree: need to arrange for inputs from those sources

- The “manual” version is again to create a tiny `main()`, which now calls constructors and initializes fields, and possibly calls other functions in the same class, before calling the function under test. Don't do it! 🤡 Use a unit testing framework

What is “Local” Shared State?

State shared only among units that are part of the same module (e.g., functions in the same class)

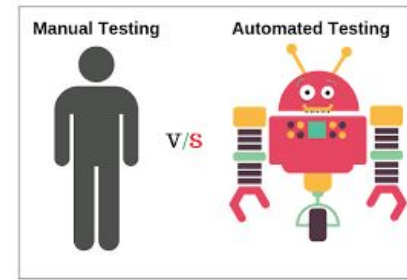
In an object-oriented language, this includes instance variables (object fields) and class variables (static fields)

“Global” shared state is application state shared with other code outside the module and environment state shared even outside the program, such as files, databases, devices, etc.

There may also be “global” inputs/outputs from/to APIs, UIs, network, or accessing the underlying platform (e.g., time of day, random number generator). “Global” shared state and inputs/outputs are usually faked during unit testing via [mocking](#)



Unit Testing Frameworks



Instead of “manual” unit testing, use some unit testing framework for your programming language, e.g., [googletest](#) for C++, [junit](#) for Java

Unit testing frameworks always provide “test runners” and usually provide facilities helpful for writing unit tests like assertions and fixtures. Unit testing frameworks often play nicely with mocking frameworks, for faking “global” inputs, outputs, and shared state

Automated testing reduces human tedium and error, and is reproducible and faster so more likely to be done frequently

➤ Automated testing refers to *running* the tests, not *writing* the tests: although templates may be provided, developers still write tests (automated test generation discussed later in course)

Test Runner

Executes a set of test groups in some order and reports results



Ordering might be built-in or specified by the tester - but test groups should not depend on each other in any way, so running in any order (or in parallel) produces exactly the same results. Intentionally dependent tests that share state should be ordered within the same test group. (Warning on terminology: What I'm calling a test group here is a JUnit test class or a GoogleTest test case or test suite)

The test runner normally does not stop when one test fails - instead runs all the tests and reports all the results, another reason test groups should be independent (why wouldn't we want to stop as soon as one test fails?)

Facilities for Writing Unit Tests

Assertions = notation for checking the results (outputs and side-effects) of executing a test.

Usually supports checking for true/false, checking whether an exception has been raised, and comparing a result to a known value. Assertions express requirements that the unit under test is expected to meet

Test fixture = *setup* code to execute before running one or more tests, *teardown* code to run after those tests, usually organized by test group. Setup can create objects, open files or network/database connections (often involves mocks), etc. Teardown destroys objects, closes files and connections, etc.



Example

Consider this variant of concat:

```
public class StringUtil {  
    public String concat(String a,String b) {  
        return a + b;  
    }  
}
```

Here is one approach to testing concat:

@Test

```
public void testStringUtil_Bad() {  
    String result = StringUtil.concat("Hello ", "World");  
    System.out.println("Result is "+result);  
}
```

Why would it be difficult to automate this test as written?



Example

An automated testing system would always treat the first test as “pass”, since there are no assertions, and a human has to check the result

Consider concat again:

```
public class StringUtil {  
    public String concat(String a,String b) {  
        return a + b;  
    }  
}
```

Here is another approach to testing concat:

```
@Test  
public void testStringUtil_Good() {  
    String result = StringUtil.concat("Hello ", "World");  
    assertEquals("Hello World", result);  
}
```

This test will “fail” if the method returns the wrong result, without human intervention



What Is Wrong With These Assertions?

```
@Test
public void getEmployeesGreaterThan25() {

    // arrange
    Employee mark = new Employee ("Mark", 19);
    Employee nina = new Employee ("Nina", 26);
    Employee jules = new Employee ("Jules", 31);
    clearDatabase();
    insertEmployee(mark);
    insertEmployee(nina);
    insertEmployee(jules);
    Predicate<Employee> constraint = e -> e.getAge() > 25;

    // act
    List<Employee> result = getEmployees(constraint);

    // assert
    assertEquals(result.size(), 2);
    assertEquals(result.get(0).getName(), "Nina");
    assertEquals(result.get(0).getAge(), 26);
    assertEquals(result.get(1).getName(), "Jules");
    assertEquals(result.get(1).getAge(), 31);
}
```

```
for (Employee e : result) {
    assertTrue(e.getAge() > 25);
}
```

```
boolean first = result.get(0).getAge() > 25;
boolean second = result.get(1).getAge() > 25;
assertTrue(first && second);
```

```
assertTrue(result.get(0).getAge() > 25);
assertTrue(result.get(1).getAge() > 25);
```

Good Assertions



Tests are software too, they may have bugs and they need to be robust, maintainable, effective and documented

Good assertions check what is required from the code under test, nothing more and nothing less

Don't write assertions which check conditions that could be split into multiple assert statements, always check the simplest condition possible

Write assertions that check requirements, not implementation details of the unit under test. Developers should not need to update a test just because the internal details of the unit under test have changed

Fixtures

A test fixture constructs a pre-defined state of a set of objects used as test inputs. The purpose of a test fixture is to ensure that there is a known and fixed environment in which tests are run so that results are predictable and repeatable.

- Preparation of input data and setup/creation of fake data
- Loading a database with a specific, known set of test data
- Copying a specific known set of test files

Good test fixtures also clean up after themselves, leave clean state for the next group of tests!

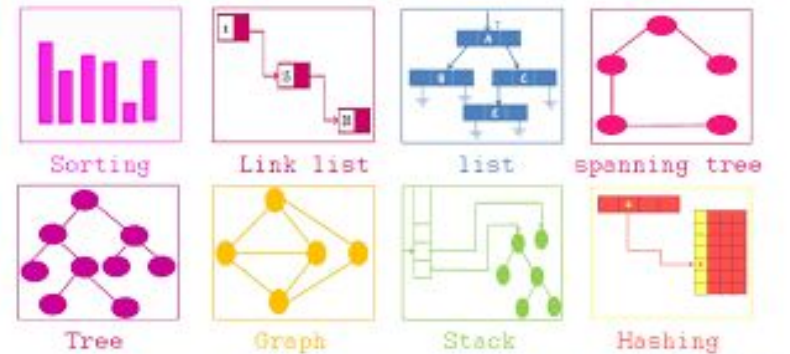


Structured Unit Testing Inputs

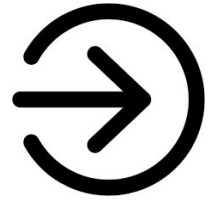
Let's say we want to test container data structures with:

- In or not in container
- Empty or full container (or min/max number of elements)
- Duplicate elements may or may not be allowed
- Ordered or organized according to some metric, or not
- Tree root node, interior node, leaf node, exactly one node, balanced vs. unbalanced, etc.

How would we write such tests?



Test **Every** Input Source



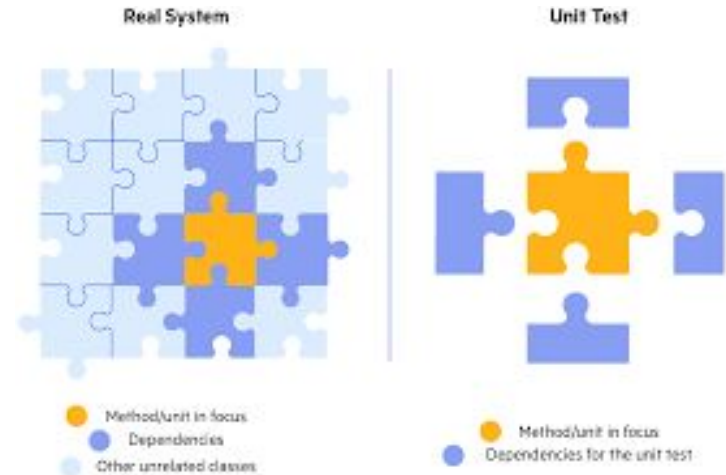
Functions typically have parameters defined in function signature and provided in function call. But parameters are not the only inputs!

- Global variables
- Return values and side-effects from library, service and other API calls are also inputs
- Data entered via GUI or CLI are also inputs
- Data retrieved from network, databases, files, devices, etc. are also inputs

The set of tests should reflect the equivalence partitions and boundaries for *all* input sources, not just parameters. Inputs from external sources are typically mocked during unit testing to ensure they indeed represent those equivalence partitions and boundaries

Agenda

1. Unit Testing
2. Test Doubles (mocking)
3. Unit Testing/Mocking demo: Madeleine Lerner
4. Deadlines



What Are Test Doubles?

The fakes used in testing are called “[test doubles](#)”

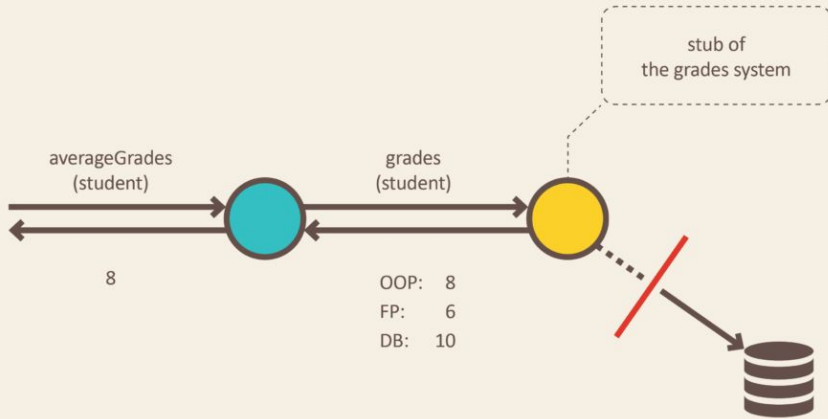
A test double is anything that replaces a production entity or service for testing purposes

Analogous to stunt doubles who perform dangerous action sequences in movies



Several Kinds of Test Doubles

Stub



Stubs support state verification, meaning they represent pre-defined hardcoded state

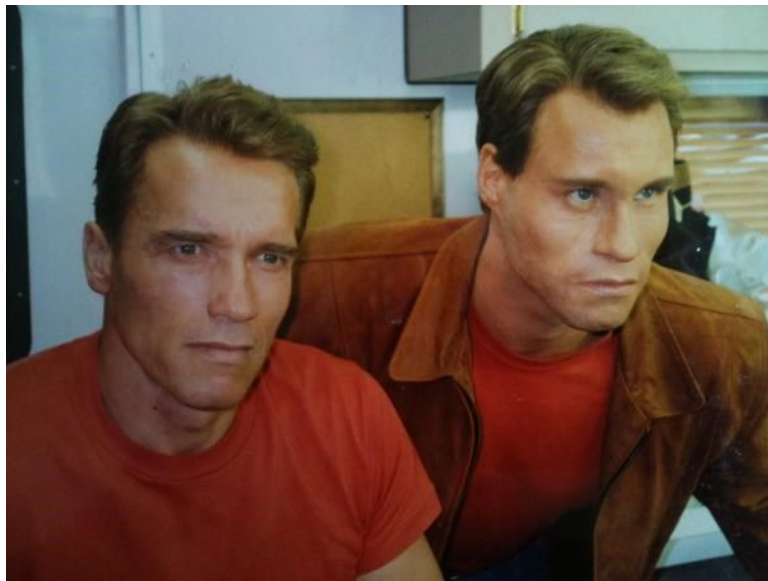
Always return the same value or always do the same thing. Might return a value that forces a specific execution path in the caller, e.g., to exercise a previously uncovered branch. To simulate a hardware failure or transaction timeout, the stub would always raise that failure or timeout exception

Dummy Objects

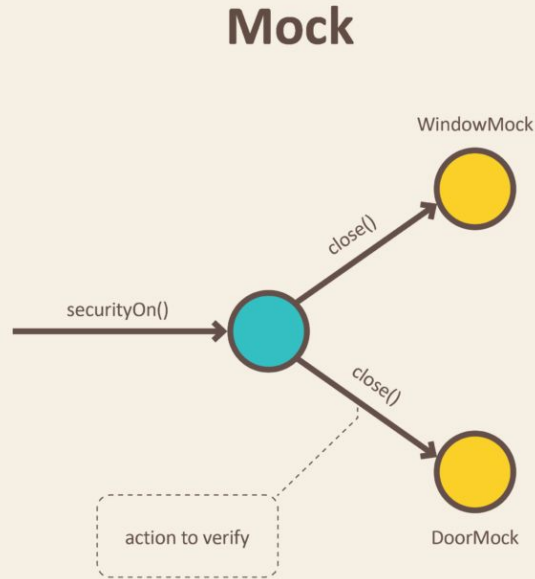
A *dummy object* is a special case of a stub that does not even return a value, like a stunt double that does no stunts (aka stand-in)

A dummy object might be passed as a mandatory parameter object that is not directly used in the test or in the code under test, but is required to call the code under test

Not the same as a **null** object, e.g., if this parameter is null then do x else do y



The Term "Mock" May Refer To Any Kind Of Test Double



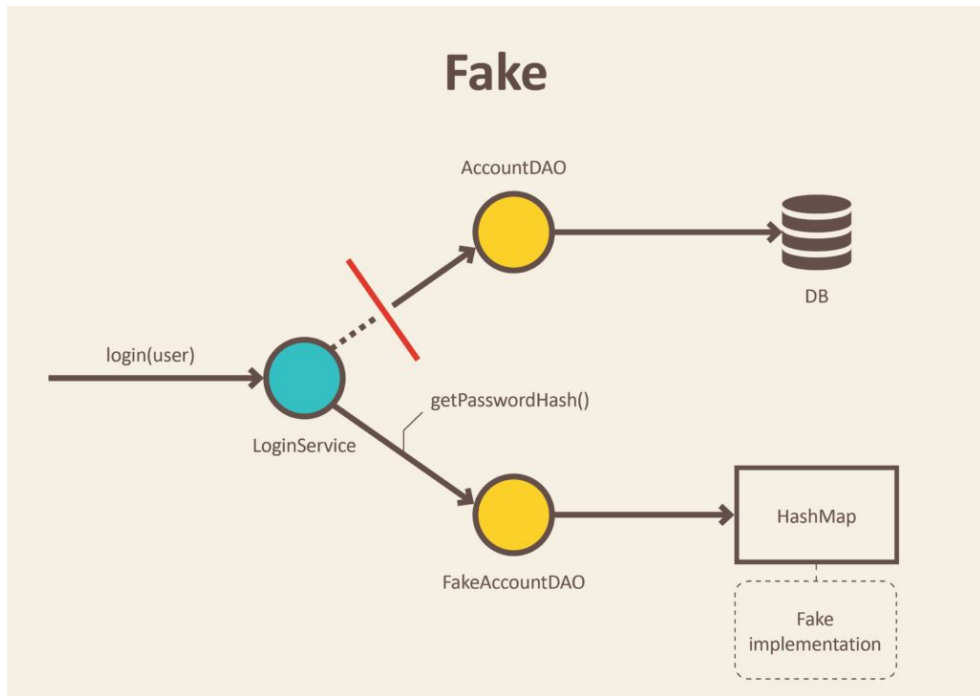
Technically, *mocks* support [behavior verification](#), meaning they actually do something (behavior). Mocks remember what they did in what order

We can use mocks to verify that all expected actions are performed in the expected order with no duplication, e.g., if tasked to 'open' a door, they won't 'open' the door again until after it has been 'closed'

Fake Objects

A *fake object* is a special case of a mock that provides a simplified and/or light-weight working implementation but is not suitable for production use, e.g., an in-memory database, a scaled-down network inaccessible by real users

Spy objects are another special case of mocks added to stubs just to record calls to the stub (the remembering part of mocks)

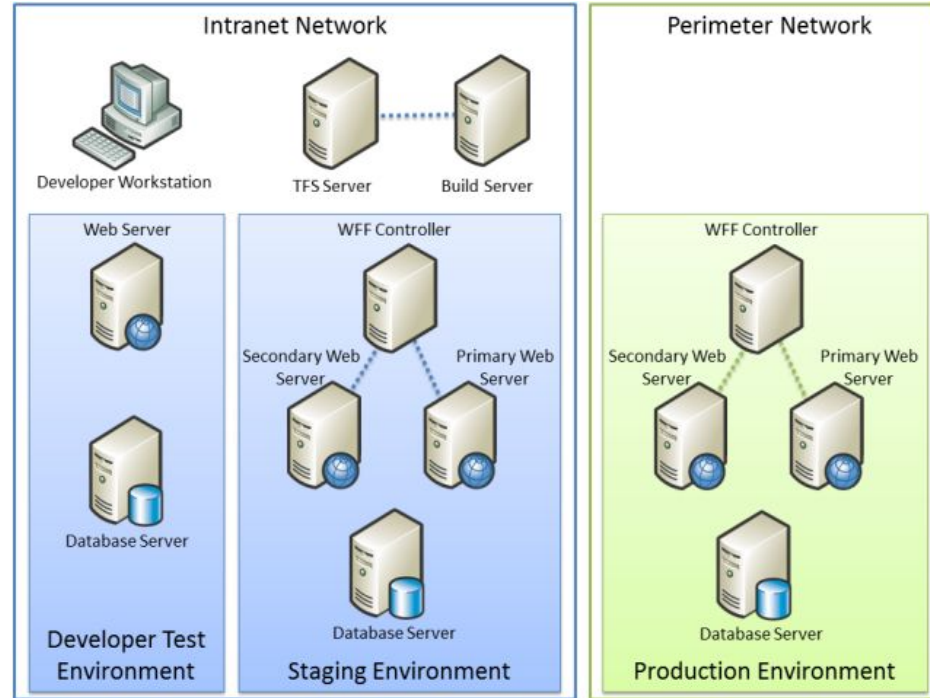


When NOT To Use Test Doubles

End-to-end system testing should use real resources, but not usually the *production* instances of those resources

End-to-end system testing for financial services does not move real money, it does not open or cancel real accounts, etc.

Imagine a summer intern or new hire using their laptop to test their modified code on the production data for your bank account...



Agenda

1. Unit Testing
2. Test Doubles
3. Unit Testing/Mocking demo: Madeleine Lerner
4. Deadlines

Keep on a straight path with proper unit testing.



Madeleine Lerner Unit Testing/Mocking Demo

[demo slides](#)



Agenda

1. Unit Testing
2. Test Doubles
3. Unit Testing/Mocking demo: Madeleine Lerner
4. Deadlines



Deadlines

[Team First Iteration](#)

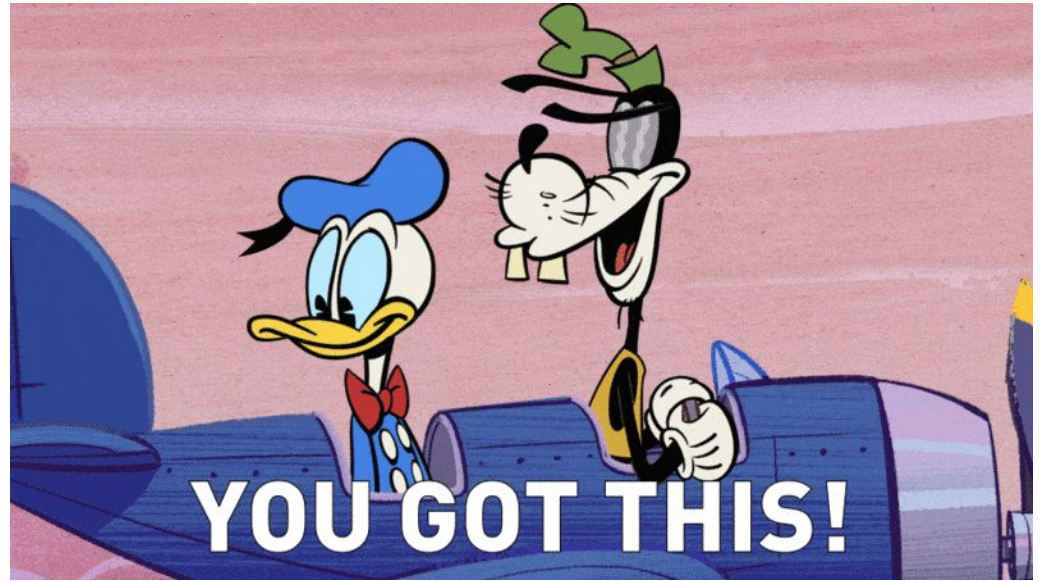
due next Friday October 18

[Team First Iteration Demo](#)

due Friday October 25

First Assessment

Friday October 25th 12:01am through Monday October 28th 11:59pm



Next Week

Integration Testing with demo

Logging

What to expect for
first assessment



Ask Me Anything

